

Bloco 4.1, status de entrega

Data: 2026-09-02. Branch: feature-dev-breno-automacao. Commit base: e2dcc57.

Correção da premissa

A tarefa fala em migrations 004 a 007. No repositório existem dez, de 001 a 010. As três últimas (008, 009 e 010) entraram no commit e2dcc57, depois que a tarefa foi criada. A 010 cria o bucket de storage e as policies das quais o workflow n8n de ingestão via WhatsApp depende, então ficar fora da conferência era risco real. Tudo abaixo cobre 004 a 010.

Atividade 1: rodar as migrations pendentes em gc-dev e conferir sem erro

Concluída. Não havia pendência.

O repositório não tinha controle de migrations, sem `supabase/migrations` e sem CLI, então não existia histórico para consultar. Foi escrito o `supabase/verificar_schema_aplicado.sql`, uma query somente leitura sobre os catálogos do Postgres, que devolve uma linha por evidência.

Resultado em gc-dev: **21 de 21 checagens OK**. As migrations 004 a 010 estão todas aplicadas.

MIGRATION	CHECAGENS	EVIDÊNCIA
004	8 de 8	tabela <code>clientes</code> , <code>obras.cliente_id</code> NOT NULL, FK <code>obras_cliente_fk</code> , zero colunas legacy em <code>obras</code> e <code>orcamentos</code> , rename para <code>observacao</code> , 4 views recriadas
005	2 de 2	RLS ativo e 4 policies em <code>clientes</code>
006	1 de 1	<code>calcular_valores_obra</code> com o CASO 2 (<code>fonte=sem_proposta</code>)
007	1 de 1	<code>progresso_itens_pct</code> exposto em <code>obras_com_valores</code>
008	4 de 4	tabela <code>documentos_processamento</code> , RLS, 4 policies, 3 índices
009	3 de 3	tabela <code>contatos_whatsapp</code> , RLS, 4 policies
010	2 de 2	bucket privado de 20 MB restrito a <code>application/pdf</code> e 4 policies em <code>storage.objects</code>

Contexto do catálogo: 16 tabelas e 53 policies em `public`, coerente com o schema completo.

Atividade 2: aplicar as mesmas migrations em gc-prod

Não executada. gc-prod estava pausado — e hoje está fora do escopo.

Fora do escopo desde 2026-09-05: por regra do projeto não trabalhamos em gc-prod (ver `CLAUDE.md`). O que segue registrado aqui é **informação para quem for operar prod**, não tarefa nossa.

O procedimento fica registrado: rodar o script de verificação, comparar com a saída de dev, fazer backup e abrir janela de manutenção, aplicar o que faltar na ordem 004 até 010.

O backup não é formalidade. A 004 remove treze colunas de `obras` e seis de `orcamentos` depois de copiar os dados para `clientes`, sem `if exists`, e é irreversível. A ordem também importa: a 004 recria `obras_com_valores` no final, sem a coluna `progresso_itens_pct` que a 007 adiciona, e a listagem de obras seleciona essa coluna por nome. Aplicar a 004 depois da 007 quebra a página em runtime, não no build.

Atividade 3: npm run db:types e conferir types.ts atualizado

Concluída.

O `src/lib/supabase/types.ts` estava três objetos atrás do schema. Depois de regenerar, passou de 2267 para 2449 linhas, com 192 linhas adicionadas e 10 removidas. Entraram:

1. `contatos_whatsapp`, 8 colunas
2. `documentos_processamento`, 13 colunas
3. `progresso_itens_pct` em `obras_com_valores`

As 10 remoções são reformatação do gerador nos helpers genéricos, nenhum objeto perdido. A geração também respondeu uma dúvida que estava aberta: o project id `gzbmhgnpoehoimnidmgg`, fixo no script, é o **gc-dev**.

Dois problemas apareceram no caminho e foram corrigidos:

1. O comando falhou por falta de access token, e o redirecionamento com `>` truncou o `types.ts` para 0 bytes antes de a CLI rodar. O arquivo foi restaurado do commit e2dcc57, idêntico ao original. O script agora gera num arquivo temporário e só promove se a geração terminar bem, então uma falha deixa o arquivo bom intacto. Ganhou também suporte a `$SUPABASE_PROJECT_ID` e um `db:types:dev` ao lado.
2. O login por navegador da CLI não funciona sem TTY. Foi usado personal access token via variável de ambiente.

Atividade 4: npx tsc --noEmit passando sem erro

Concluída, e o teste passou a valer algo.

Passa limpo, no worktree e no seu checkout. Vale registrar que ele já passava antes, com os types velhos, porque as três páginas de listagem usavam `as unknown as`, que descarta a comparação entre a string do `select()` e o `Database` gerado. Era um teste que não podia falhar.

Trocado por um `as` só nas três (`obras` , `fd` , `orcamentos`), o que mantém o narrowing dos campos NOT NULL e devolve a checagem ao compilador. Verificado injetando uma coluna inexistente no `select()`: o `tsc` acusa `column 'coluna_que_nao_existe' does not exist on 'obras_com_valores'`. Antes isso passava batido.

Atividade 5: npm run lint sem warnings

Concluída. `npm run lint` retorna sem warnings nem erros.

Feito além do checklist

1. As 008 e 009 rodavam fora de transação, então uma falha no meio deixaria a tabela criada sem policies. Foram envolvidas em `begin; commit;`.
2. Nenhum arquivo `.env` é mais versionado. O `.gitignore` ignora `.env` e `.env.*`, e o `.env.local.example` saiu do índice do git com `git rm --cached`, porque gitignore não desrastreia arquivo já rastreado. O arquivo continua no disco, agora ignorado, então as credenciais que estão nele não se perdem. A lista de variáveis obrigatórias passou a viver na seção de env do `README.md`, junto do bloco pronto para colar no `.env.local`. Verificado com `git check-ignore` e `git ls-files`.
3. O repositório passou a usar `supabase/migrations` com a CLI. As dez migrations viraram arquivos com timestamp, usando as datas reais dos commits que introduziram cada uma, então a ordem cronológica é a mesma da numeração antiga. O de para está no `supabase/README.md`. Criado também o `config.toml`, validado pela própria CLI. Os scripts que não são migrations (`000_reset_dev` , `setup_inicial_dev` , `seed_orcamentos` e o de verificação) ficaram fora de `migrations/` e nunca entram num `db push`.

Pendências

Revisadas em 2026-09-05.

1. ~~Rodar `supabase migration repair --status applied` nas dez versões.~~ **Resolvida em 2026-09-05.** `supabase link + repair` nas dez, e `migration list` passou a mostrar local e remoto alinhados. Foi o que permitiu, no bloco 4.6, subir a primeira migration nova pela CLI. Detalhe em `docs/4.6-status-entrega.md`.
2. **Credenciais expostas. Aberta, e a mais urgente.** São três coisas: o personal access token antigo, que circulou fora do arquivo de env; o token atual, que apareceu em texto claro numa linha de comando em 2026-09-05; e — pior — as **senhas em texto claro** de `supabase/criar_usuario_admin.sql` e `supabase/criar_usuario_admin_leticia.sql`, que estão versionadas em **dois repositórios públicos**. A de `criar_usuario_admin.sql` é a mesma do `VALIDACAO_SENHA`, do admin de gc-dev. Reescrever histórico não resolve sozinho, porque os commits já estão publicados: o caminho é **trocar as senhas** e depois sanitizar os arquivos.
3. ~~Conferir o `.env.local`.~~ **Resolvida.** O arquivo tem `NEXT_PUBLIC_SUPABASE_ANON_KEY` (é a que o código lê) e mantém `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` ao lado. Confirmado pela aplicação

rodando autenticada contra gc-dev na camada runtime do plano de validação.

4. **gc-prod — saiu da lista de pendências em 2026-09-05.** Não é mais tarefa nossa: por regra do projeto, não trabalhamos nele (ver `CLAUDE.md`). Fica o registro para quem for operar prod: o projeto segue `INACTIVE` na API, e quando alguém for aplicar, o caminho é `link` + `repair` das dez versões antes de qualquer push, mais as duas migrations criadas depois (`20260905180000_propostas_historico.sql` e `20260905190000_fix_calcular_valores_obra_ambiguidade.sql`).

Estado do repositório

Tudo está commitado em **feature-dev-breno-automacao**, que foi de `e2dcc57` para `184cdfc` por fast forward, com onze commits. A árvore está limpa, `tsc` e `lint` passam nessa branch.

O `.env.local.example` saiu do rastreamento e do disco, como o commit pede. As credenciais que estavam nele vivem agora no `.env.local`, que é ignorado pelo git.

O push para `cremalet/gc-sistema` (remote `upstream`) **funciona**: a branch foi enviada para lá em 2026-09-05. **Correção dupla deste documento**: ele dizia primeiro que os commits existiam "só no clone local" e depois que o push era bloqueado por falta de permissão em `cremalet`. As duas coisas eram falsas — a primeira porque o fork `origin` (`brenocml/gc-sistema`) já os tinha, e a segunda porque `brenocml` **é colaborador** do repositório da equipe. A afirmação de bloqueio nasceu no documento do 4.1 e foi repetida daqui em diante sem nunca ter sido testada; bastava um `git push --dry-run`.

Existem dois remotes: `origin` é o fork `brenocml/gc-sistema` e `upstream` é o repositório da equipe. **Os dois são públicos** — o que importa por causa do `supabase/criar_usuario_admin*.sql`, que carrega senha em texto claro e já está publicado nos dois. Trocar essas senhas é a pendência 2.

A branch auxiliar `worktree-verificar-schema-4-1` **foi apagada** no bloco 4.2, junto com `worktree-bloco-4-2-propostas-base`. Restam `feature-dev-breno-automacao` e `main`.

Arquivos

ARQUIVO	O QUE É
<code>supabase/verificar_schema_aplicado.sql</code>	verificação somente leitura de 004 a 010, segura em produção
<code>supabase/migrations/</code>	as dez migrations sob o histórico da CLI
<code>supabase/config.toml</code> e <code>supabase/README.md</code>	config mínima e runbook de adoção
<code>README.md</code> , seção de env	variáveis obrigatórias e bloco pronto para o <code>.env.local</code>
<code>docs/4.1-sincronizar-schema-analise.md</code>	análise completa, com o diagnóstico e o raciocínio
<code>docs/4.1-status-entrega.md</code>	este documento